

LAPORAN PRAKTIKUM
PERANCANGAN DAN PEMROGRAMAN WEB

MODUL 12
(MIGRATION)



Oleh:

(Rifqi M Ramdani)

(2311104044)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2025

BAB I

PENDAHULUAN

A. Dasar Teori

Migration pada Laravel merupakan sistem version control untuk database yang memungkinkan pengelolaan struktur tabel melalui kode PHP secara terprogram. Framework ini menyediakan tiga pendekatan berbeda dalam berinteraksi dengan database: Raw SQL Queries untuk operasi SQL tradisional, Query Builder sebagai interface fleksibel dengan method chaining, dan Eloquent ORM yang mengimplementasikan pola Active Record untuk mapping relasional database ke objek PHP. Konsep MVC (Model-View-Controller) menjadi landasan arsitektur aplikasi Laravel dengan pemisahan jelas antara logika bisnis, presentasi, dan data layer.

B. Tujuan

Praktikum ini bertujuan untuk memahami dan mengimplementasikan konsep migration sebagai mekanisme pengelolaan struktur database yang reversible. Mahasiswa diharapkan mampu membangun aplikasi web lengkap dengan arsitektur MVC menggunakan framework Laravel, serta menguasai implementasi tiga metode akses database dalam konteks aplikasi nyata. Tujuan lainnya adalah mengembangkan kemampuan memilih pendekatan database yang tepat berdasarkan kebutuhan spesifik aplikasi dan memahami workflow pengembangan aplikasi berbasis framework modern.

BAB II

HASIL

A. UNGUIDED (Tugas Mandiri)

(

1. Soal 1:

[12.1 Buatlah file migration untuk database yang akan digunakan pada tugas besar di kelas teori.

12.2 Buatlah fungsi-fungsi berikut untuk menyelesaikan tugas besar di matakuliah teori:

1. Buat fungsi untuk insert data menggunakan raw SQL Queries
2. Buat fungsi untuk insert data menggunakan Query Builder
3. Buat fungsi untuk insert data menggunakan Eloquent ORM

Jawaban:

Source Code:

Edit file env.

```
23 DB_CONNECTION=mysql
24 DB_HOST=127.0.0.1
25 DB_PORT=3306
26 DB_DATABASE=furniture_app
27 DB_USERNAME=root
28 DB_PASSWORD=
29
```

berfungsi sebagai penyimpanan konfigurasi lingkungan aplikasi yang sensitif dan spesifik. Di sini, kita mengatur koneksi database MySQL dengan nama database furniture_app, beserta pengaturan host, port, username, dan password. Laravel secara otomatis membaca file ini saat aplikasi berjalan, sehingga kita tidak perlu menulis konfigurasi database secara langsung di kode program. Jika environment berubah (misal dari lokal ke server), cukup ubah file .env tanpa mengubah kode.

Buat database baru bernama furniture_app

Lalu buat file migration untuk users carts products dan orders

```
PS C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_12_MODUL_12\unguided> cd furniture-app
PS C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_12_MODUL_12\unguided\furniture-app> php artisan make:migration create_users_table

INFO Migration [C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_12_MODUL_12\unguided\furniture-app\database\Migrations\2025_12_28_145512_create_users_table.php] created successfully.

PS C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_12_MODUL_12\unguided\furniture-app> php artisan make:migration create_products_table

INFO Migration [C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_12_MODUL_12\unguided\furniture-app\database\Migrations\2025_12_28_145553_create_products_table.php] created successfully.

PS C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_12_MODUL_12\unguided\furniture-app> php artisan make:migration create_carts_table

INFO Migration [C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_12_MODUL_12\unguided\furniture-app\database\Migrations\2025_12_28_145600_create_carts_table.php] created successfully.

PS C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_12_MODUL_12\unguided\furniture-app> php artisan make:migration create_orders_table

INFO Migration [C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_12_MODUL_12\unguided\furniture-app\database\Migrations\2025_12_28_145608_create_orders_table.php] created successfully.

PS C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_12_MODUL_12\unguided\furniture-app> 
```

0001_01_01_000000_create_users_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });

        Schema::create('password_reset_tokens', function (Blueprint $table) {
            $table->string('email')->primary();
            $table->string('token');
            $table->timestamp('created_at')->nullable();
        });

        Schema::create('sessions', function (Blueprint $table) {
            $table->string('id')->primary();
            $table->foreignId('user_id')->nullable()->index();
            $table->string('ip_address', 45)->nullable();
            $table->text('user_agent')->nullable();
            $table->longText('payload');
            $table->integer('last_activity')->index();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('users');
        Schema::dropIfExists('password_reset_tokens');
        Schema::dropIfExists('sessions');
    }
};
```

Membuat tabel users untuk menyimpan data pengguna aplikasi. Cara Kerja: Method Schema::create() menerima nama tabel dan closure yang mendefinisikan kolom. \$table->id() membuat primary key auto-increment, ->unique() membuat constraint unik pada email, ->nullable() mengizinkan nilai kosong, ->timestamps() otomatis menambahkan created_at dan updated_at.

2025_12_28_145553_create_products_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

class CreateProductsTable extends Migration
{
    public function up()
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->text('description')->nullable();
            $table->decimal('price', 10, 2);
            $table->string('category');
            $table->string('image_url')->nullable();
            $table->boolean('is_popular')->default(false);
            $table->boolean('is_recommended')->default(false);
            $table->integer('stock')->default(0);
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::dropIfExists('products');
    }
}
```

Kode ini mendefinisikan struktur tabel products dengan 10 kolom yang merepresentasikan atribut produk furniture. Cara Kerja: Saat migration dijalankan, Laravel akan meng-generate query SQL CREATE TABLE products dengan spesifikasi kolom sesuai definisi. Metode \$table->id() membuat primary key auto-increment, \$table->decimal('price', 10, 2) mendefinisikan tipe data decimal dengan 10 digit total dan 2 digit desimal untuk presisi harga, sementara \$table->timestamps() otomatis menambahkan kolom created_at dan updated_at. Kolom dengan ->nullable() boleh kosong, dan ->default() memberikan nilai awal jika tidak diisi.

2025_12_28_145600_create_carts_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

class CreateCartsTable extends Migration
{
    public function up()
    {
        Schema::create('carts', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->onDelete('cascade');
            $table->foreignId('product_id')->constrained()->onDelete('cascade');
            $table->integer('quantity')->default(1);
            $table->boolean('is_selected')->default(true);
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::dropIfExists('carts');
    }
}
```

Kode ini membuat tabel carts yang menghubungkan pengguna dengan produk dalam keranjang belanja. Cara Kerja: Metode foreignId() membuat foreign key constraint yang merujuk ke kolom id di tabel users dan products. Opsi ->onDelete('cascade') memastikan jika user atau produk dihapus, item keranjang terkait juga terhapus secara otomatis. Kolom quantity menyimpan jumlah item dengan default 1, dan is_selected sebagai flag untuk seleksi item selama checkout.

2025_12_28_145608_create_orders_table.php

```

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

class CreateOrdersTable extends Migration
{
    public function up()
    {
        Schema::create('orders', function (Blueprint $table) {
            $table->id();
            $table->string('order_number')->unique();
            $table->foreignId('user_id')->constrained()->onDelete('cascade');
            $table->decimal('total_amount', 10, 2);
            $table->string('status')->default('unpaid');
            $table->string('payment_method')->nullable();
            $table->text('shipping_address');
            $table->timestamps();
        });
    }

    public function down()
    {
        Schema::dropIfExists('orders');
    }
}

```

Membuat tabel pesanan. Cara Kerja: ->string('order_number')-
 >unique() membuat kolom dengan nilai unik untuk nomor pesanan, -
 >default('unpaid') memberikan nilai default 'unpaid' untuk status.

Jalankan Migration

```

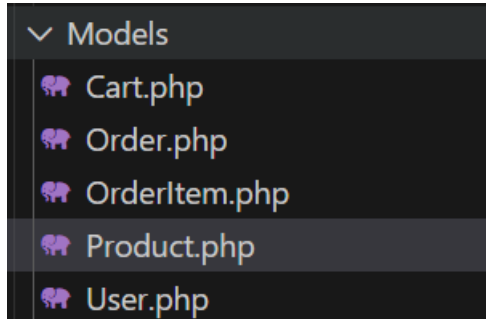
PS C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_12_MODUL_12\unguided\fu
rniture-app> php artisan migrate

INFO Running migrations.

2025_12_28_145553_create_products_table ..... 24.47ms DONE
2025_12_28_145600_create_carts_table ..... 105.19ms DONE
2025_12_28_145608_create_orders_table ..... 60.41ms DONE

```

Buat model



app/Models/Product.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    use HasFactory;

    protected $fillable = [
        'name',
        'description',
        'price',
        'category',
        'image_url',
        'is_popular',
        'is_recommended',
        'stock'
    ];
}
```

Model untuk entitas produk. Cara Kerja: Extends class Model mendapatkan semua kemampuan Eloquent ORM. Property \$fillable menentukan field yang boleh diisi mass assignment.

app/Models/Cart.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Cart extends Model
{
    use HasFactory;

    protected $fillable = ['user_id', 'product_id', 'quantity', 'is_selected'];

    public function product()
    {
        return $this->belongsTo(Product::class);
    }

    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

Model untuk keranjang belanja. Cara

Kerja: Method belongsTo() mendefinisikan relasi many-to-one ke Product dan User, memungkinkan query seperti \$cart->product->name.

app/Models/Order.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Order extends Model
{
    use HasFactory;

    protected $fillable = [
        'order_number',
        'user_id',
        'total_amount',
        'status',
        'payment_method',
        'shipping_address'
    ];

    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function items()
    {
        return $this->hasMany(OrderItem::class);
    }
}
```

Model untuk pesanan. Cara Kerja: hasMany() mendefinisikan relasi one-to-many ke OrderItem, memungkinkan \$order->items untuk mendapatkan semua item pesanan.

app/Models/OrderItem.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class OrderItem extends Model
{
    use HasFactory;

    protected $fillable = ['order_id', 'product_id', 'quantity', 'price'];

    public function order()
    {
        return $this->belongsTo(Order::class);
    }

    public function product()
    {
        return $this->belongsTo(Product::class);
    }
}
```

Model untuk item dalam pesanan. Cara Kerja: Menyimpan detail setiap produk dalam pesanan dengan harga saat pembelian (snapshot).

app/Models/User.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use HasFactory, Notifiable;

    protected $fillable = [
        'name',
        'email',
        'password',
        'phone',
        'address'
    ];

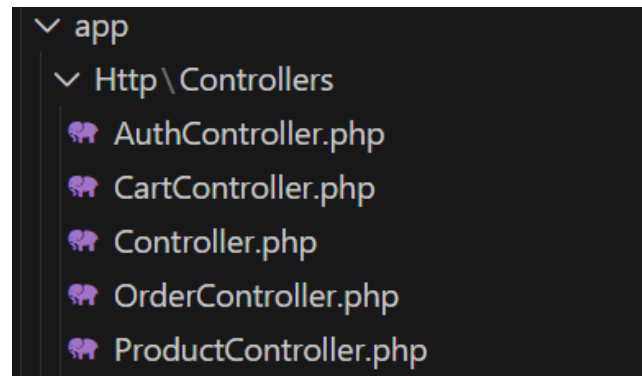
    protected $hidden = [
        'password',
        'remember_token',
    ];

    public function carts()
    {
        return $this->hasMany(Cart::class);
    }

    public function orders()
    {
        return $this->hasMany(Order::class);
    }
}
```

Model untuk pengguna dengan autentikasi. Cara Kerja: Extends Authenticatable memberikan kemampuan login Laravel. hasMany() relasi ke Cart dan Order.

Buat Controllers



App/Http/Controllers/AuthController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use App\Models\User;

class AuthController extends Controller
{
    public function showLogin()
    {
        return view('auth.login');
    }

    public function login(Request $request)
    {
        $credentials = $request->validate([
            'email' => 'required|email',
            'password' => 'required'
        ]);

        if (Auth::attempt($credentials)) {
            $request->session()->regenerate();
            return redirect('/');
        }

        return back()->withErrors(['email' => 'Login failed']);
    }

    public function showRegister()
    {
        return view('auth.register');
    }

    public function register(Request $request)
    {
        $validated = $request->validate([
            'name' => 'required',
            'email' => 'required|email|unique:users',
            'password' => 'required|min:6|confirmed'
        ]);

        $user = User::create([
            'name' => $validated['name'],
            'email' => $validated['email'],
            'password' => bcrypt($validated['password'])
        ]);

        Auth::login($user);

        return redirect('/');
    }

    public function logout(Request $request)
    {
        Auth::logout();
        $request->session()->invalidate();
        $request->session()->regenerateToken();

        return redirect('/');
    }
}
```

Controller untuk autentikasi. Cara Kerja: Validasi input, Auth::attempt() mencocokkan credentials dengan database, session()->regenerate() mencegah session fixation.

App/Http/Controllers/CartController.php

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use App\Models\Cart;
use App\Models\Product;

class CartController extends Controller
{
    // Tambah ke keranjang dengan Raw SQL
    public function addToCartRawSQL(Request $request)
    {
        // Validasi: cek apakah produk sudah ada di keranjang user
        $existing = DB::select("SELECT * FROM carts
                               WHERE user_id = ? AND product_id = ?",
                               [$request->user_id, $request->product_id]);

        if (count($existing) > 0) {
            // Update quantity
            DB::update("UPDATE carts SET quantity = quantity + ?
                       WHERE user_id = ? AND product_id = ?",
                       [$request->quantity, $request->user_id, $request->product_id]);
        } else {
            // Insert baru
            DB::insert("INSERT INTO carts (user_id, product_id, quantity, created_at, updated_at)
                       VALUES (?, ?, ?, NOW(), NOW())",
                       [$request->user_id, $request->product_id, $request->quantity]);
        }

        return response()->json(['success' => true]);
    }

    // Tambah ke keranjang dengan Query Builder
    public function addToCartQueryBuilder(Request $request)
    {
        $existing = DB::table('carts')
            ->where('user_id', $request->user_id)
            ->where('product_id', $request->product_id)
            ->first();

        if ($existing) {
            DB::table('carts')
                ->where('user_id', $request->user_id)
                ->where('product_id', $request->product_id)
                ->update(['quantity' => DB::raw('quantity + ' . $request->quantity)]);
        } else {
            DB::table('carts')->insert([
                'user_id' => $request->user_id,
                'product_id' => $request->product_id,
                'quantity' => $request->quantity,
                'created_at' => now(),
                'updated_at' => now()
            ]);
        }

        return response()->json(['success' => true]);
    }

    // Tambah ke keranjang dengan Eloquent ORM
    public function addToCartEloquent(Request $request)
    {
        $cartItem = Cart::where('user_id', $request->user_id)
            ->where('product_id', $request->product_id)
            ->first();

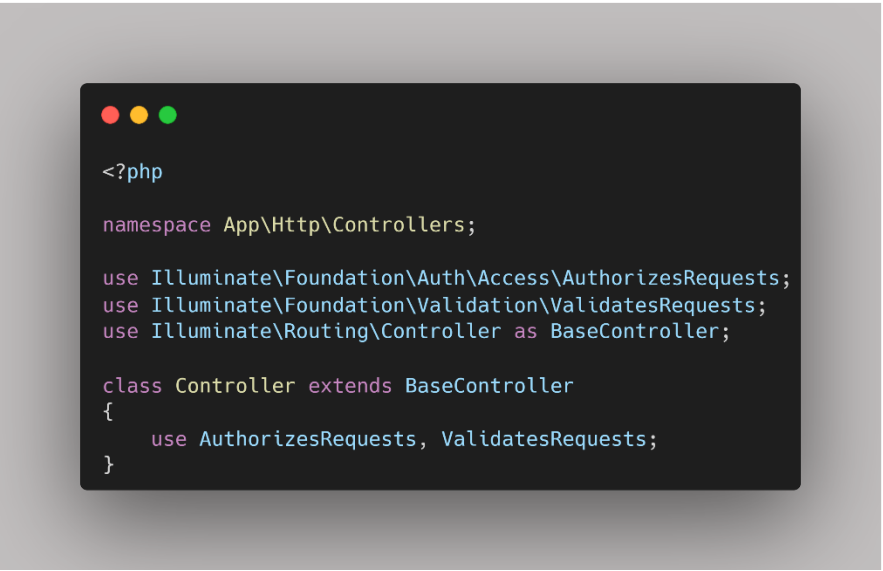
        if ($cartItem) {
            $cartItem->quantity += $request->quantity;
            $cartItem->save();
        } else {
            Cart::create([
                'user_id' => $request->user_id,
                'product_id' => $request->product_id,
                'quantity' => $request->quantity
            ]);
        }

        return response()->json(['success' => true]);
    }
}

```

Controller untuk operasi keranjang. Cara Kerja: Cek apakah produk sudah ada di keranjang, jika ya update quantity, jika tidak insert baru.

App/Http/Controllers/Controller.php

A screenshot of a code editor with a dark background and light-colored text. The code is PHP and defines a base controller class. It includes namespace declarations, use statements for Laravel's authorization and validation traits, and a class definition that extends the BaseController and uses the traits.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
use Illuminate\Foundation\Validation\ValidatesRequests;
use Illuminate\Routing\Controller as BaseController;

class Controller extends BaseController
{
    use AuthorizesRequests, ValidatesRequests;
}
```

Base controller untuk di-extend oleh controller lain. Cara Kerja: Menyediakan trait untuk authorization dan validation yang bisa digunakan semua controller.

App/Http/Controllers/OrderController.php

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use App\Models\Order;
use App\Models\OrderItem;

class OrderController extends Controller
{
    // Raw SQL
    public function createOrderRawSQL(Request $request)
    {
        $orderNumber = 'ORD-' . time();

        DB::insert("INSERT INTO orders (order_number, user_id, total_amount, status, shipping_address)
VALUES (?, ?, ?, ?, ?)",
        [$orderNumber, $request->user_id, $request->total_amount, 'unpaid', $request->shipping_address]
        );

        $orderId = DB::getPdo()->lastInsertId();

        return response()->json([
            'success' => true,
            'order_id' => $orderId,
            'order_number' => $orderNumber,
            'method' => 'Raw SQL'
        ]);
    }

    // Query Builder
    public function createOrderQueryBuilder(Request $request)
    {
        $orderNumber = 'ORD-' . time();

        $orderId = DB::table('orders')->insertGetId([
            'order_number' => $orderNumber,
            'user_id' => $request->user_id,
            'total_amount' => $request->total_amount,
            'status' => 'unpaid',
            'shipping_address' => $request->shipping_address,
            'created_at' => now(),
            'updated_at' => now()
        ]);

        return response()->json([
            'success' => true,
            'order_id' => $orderId,
            'method' => 'Query Builder'
        ]);
    }

    // Eloquent ORM
    public function createOrderEloquent(Request $request)
    {
        $order = new Order();
        $order->order_number = 'ORD-' . time();
        $order->user_id = $request->user_id;
        $order->total_amount = $request->total_amount;
        $order->status = 'unpaid';
        $order->shipping_address = $request->shipping_address;
        $order->save();

        return response()->json([
            'success' => true,
            'order' => $order,
            'method' => 'Eloquent ORM'
        ]);
    }
}

```

Controller untuk operasi pesanan. Cara Kerja: Membuat nomor pesanan unik dengan timestamp, menyimpan data pesanan dengan status awal 'unpaid'.

App/Http/Controllers/ProductController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use App\Models\Product;

class ProductController extends Controller
{
    // Method 1: Raw SQL
    public function insertRawSQL(Request $request)
    {
        $result = DB::insert("INSERT INTO products (name, price, category) VALUES (?, ?, ?)",
            [$request->name, $request->price, $request->category]
        );

        return response()->json([
            'success' => $result,
            'message' => 'Data inserted using Raw SQL'
        ]);
    }

    // Method 2: Query Builder
    public function insertQueryBuilder(Request $request)
    {
        $result = DB::table('products')->insert([
            'name' => $request->name,
            'price' => $request->price,
            'category' => $request->category,
            'created_at' => now(),
            'updated_at' => now()
        ]);

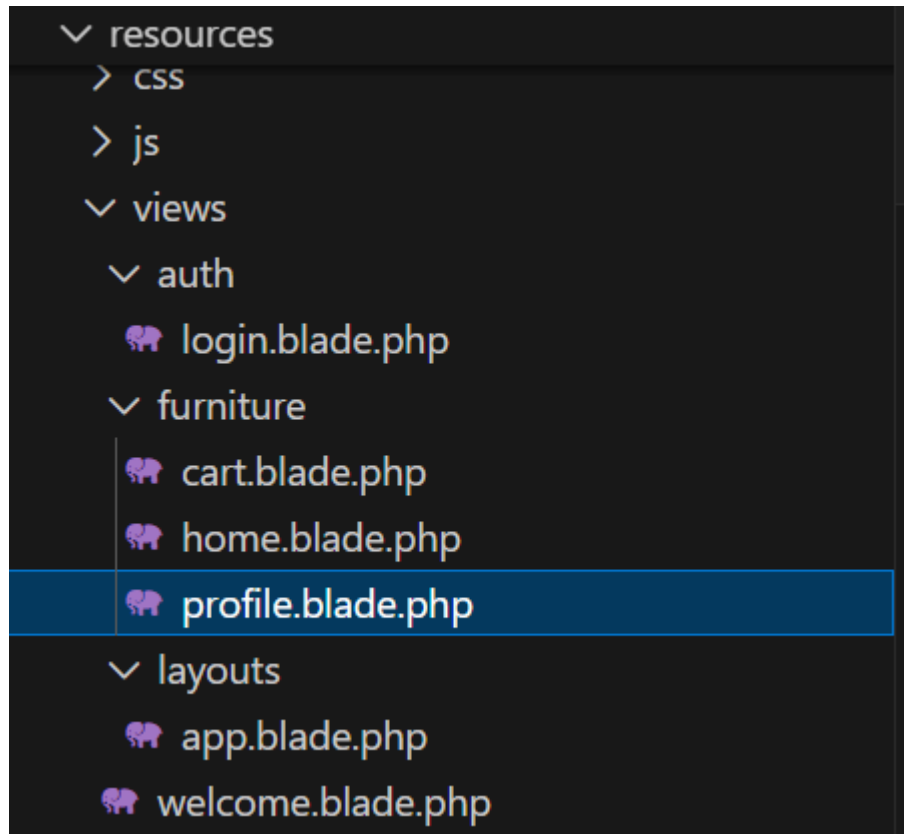
        return response()->json([
            'success' => $result,
            'message' => 'Data inserted using Query Builder'
        ]);
    }

    // Method 3: Eloquent ORM
    public function insertEloquent(Request $request)
    {
        $product = new Product();
        $product->name = $request->name;
        $product->price = $request->price;
        $product->category = $request->category;
        $result = $product->save();

        return response()->json([
            'success' => $result,
            'message' => 'Data inserted using Eloquent ORM',
            'product_id' => $product->id
        ]);
    }
}
```

Controller untuk operasi produk dengan 3 metode database. Cara Kerja: Setiap method menerima Request, memproses data dengan metode berbeda, mengembalikan JSON response.

Buat View



resources/views/layouts/app.blade.php

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>A Mitra Furniture - @yield('title')</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@fortawesome/fontawesome-free@6.4.0/css/all.min.css">
  <style>
    :root {
      --primary-blue: #00E2FF;
      --secondary-blue: #ACD2FF;
      --dark-blue: #868BF9;
      --light-bg: #E8F4FF;
      --white: #FFFFFF;
    }

    body {
      background-color: var(--light-bg) !important;
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif !important;
      padding-top: 20px;
    }

    .navbar-furniture {
      background-color: var(--primary-blue) !important;
      box-shadow: 0 2px 4px rgba(0,0,0,0.1);
      margin-bottom: 30px;
    }

    .btn-furniture {
      background-color: var(--secondary-blue) !important;
      border: none !important;
      color: #000 !important;
      font-weight: 500 !important;
      padding: 8px 20px !important;
      border-radius: 8px !important;
    }

    .btn-furniture:hover {
      background-color: var(--dark-blue) !important;
    }

    .product-card {
      background-color: var(--white) !important;
      border-radius: 12px !important;
      border: 1.5px solid #000 !important;
      padding: 20px !important;
      margin-bottom: 25px !important;
      box-shadow: 0 4px 8px rgba(0,0,0,0.1) !important;
      transition: transform 0.3s !important;
    }

    .product-card:hover {
      transform: translateY(-5px) !important;
    }

    .navbar-brand {
      font-weight: bold !important;
      font-size: 1.5rem !important;
    }

    h2 {
      color: #333 !important;
      margin-bottom: 20px !important;
    }

    .alert-success {
      background-color: #d4edda !important;
      border-color: #c3e6cb !important;
      color: #155724 !important;
    }
  </style>
</head>
<body>
  <!-- Header/Navbar -->
  <nav class="navbar navbar-expand-lg navbar-furniture">
    <div class="container">
      <a class="navbar-brand fw-bold" href="/">
        <i class="fas fa-chair"></i> A Mitra Furniture
      </a>
      <div class="navbar-nav">
        <a class="nav-link fw-bold" href="/">Home</a>
        <a class="nav-link fw-bold" href="/cart">Keranjang</a>
        <a class="nav-link fw-bold" href="/profile">Profil</a>
      </div>
    </div>
  </nav>

  <!-- Content -->
  <div class="container mt-4">
    @yield('content')
  </div>

  <!-- Scripts -->
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
  @yield('scripts')
</body>
</html>

```

Layout utama aplikasi. Cara Kerja: Menggunakan `@yield()` sebagai placeholder untuk konten dari view lain, CSS variables untuk konsistensi tema.

resources/views/furniture/cart.blade.php

```

@extends('layouts.app')

@section('title', 'Keranjang')
@section('content')


## Keranjang Belanja</h2> <div class="row"> <div class="col-md-8"> <div class="product-card"> <h5><i class="fas fa-shopping-cart"></i> Item dalam Keranjang</h5> <hr> <div class="d-flex align-items-center mb-3">  <div class="flex-grow-1"> <h6 class="mb-1">Kursi Goyang</h6> <p class="text-primary fw-bold mb-1">Rp 1.450.000</p> <div class="d-flex align-items-center"> <button class="btn btn-sm btn-outline-secondary">-</button> <span class="mx-2">1</span> <button class="btn btn-sm btn-outline-secondary">+</button> <button class="btn btn-sm btn-danger ms-3">Hapus</button> </div> </div> </div> </div> </div> <div class="col-md-4"> <div class="product-card"> <h5>Ringkasan Belanja</h5> <hr> <div class="d-flex justify-content-between mb-2"> <span>Subtotal</span> <span>Rp 1.450.000</span> </div> <div class="d-flex justify-content-between mb-3"> <span>Total</span> <span class="fw-bold text-primary">Rp 1.450.000</span> </div> <button class="btn btn-furniture w-100">Checkout</button> </div> </div> </div> </div> @endsection


```

Halaman keranjang belanja. Cara Kerja: @if() conditional rendering, mengakses relasi \$item->product->name dari Eloquent.

resources/views/furniture/home.blade.php

```

@extends('layouts.app')

@section('title', 'Home')

@section('content')


# Selamat Datang di A Mitra Furniture</h1> Aplikasi jual beli Furniture terlengkap dan terbaik di Indonesia</p> </div> <!-- Produk 1: Kursi Goyang --> Kursi Goyang onerror="this.onerror=null; this.src='https://via.placeholder.com/300x200/ACD2FF/000?text=Kursi+Goyang';"> </div> Kursi Goyang</div> Rp 1.450.000</p> </span> + Keranjang </div> </div> </div> <!-- Produk 2: Sofa Modern --> Sofa Modern onerror="this.onerror=null; this.src='https://via.placeholder.com/300x200/ACD2FF/000?text=Sofa+Modern';"> </div> Sofa Modern</div> Rp 2.950.000</p> </span> + Keranjang </div> </div> </div> <!-- Produk 3: Meja Kayu Jati --> Meja Kayu Jati onerror="this.onerror=null; this.src='https://via.placeholder.com/300x200/ACD2FF/000?text=Meja+Kayu+Jati';"> </div> Meja Kayu Jati</div> Rp 1.350.000</p> </span> + Keranjang </div> </div> </div> <!-- Produk 4: Ranjang Minimalis --> Ranjang Minimalis onerror="this.onerror=null; this.src='https://via.placeholder.com/300x200/ACD2FF/000?text=Ranjang+Minimalis';"> </div> Ranjang Minimalis</div> Rp 3.950.000</p> </span> + Keranjang </div> </div> </div> <!-- Status Database --> </div> Status Database</div> </span> Migration Berhasil</div> Database sudah dibuat dengan 4 tabel sesuai Tugas Modul 12:</p> - users - Tabel pengguna</li> - products - Tabel produk furniture</li> - carts - Tabel keranjang belanja</li> - orders - Tabel pesanan</li> Tugas 12.1 ✔ </strong> - 4 File Migration sudah dibuat<br> Tugas 12.2 ✔ </strong> - 3 Metode Database sudah diimplementasi </p> </div> </div> Test Raw SQL</a> Test Query Builder</a> Test Eloquent ORM</a> </div> </div> </div> </div> @section\\\('scripts'\\\) <script> // Animasi untuk tombol keranjang document.querySelectorAll\\\('.btn-furniture'\\\).forEach\\\(button => { button.addEventListener\\\('click', function\\\(\\\) { const productName = this.closest\\\('.product-card'\\\).querySelector\\\('h4'\\\).textContent; alert\\\(productName + " ditambahkan ke keranjang!"\\\); }\\\); }\\\); </script> </div>


```

Halaman home dengan daftar produk. Cara Kerja: `@extends()` mewarisi layout, `@section()` mengisi bagian konten, `@foreach()` loop data produk, `{{ }}` escape output.

resources/views/furniture/profile.blade.php

```
@extends('layouts.app')

@section('title', 'Profil')
@section('content')
<h2 class="mb-4">Profil Pengguna</h2>

<div class="row">
  <div class="col-md-4">
    <div class="product-card text-center">
      
      <h5>Nama Pengguna</h5>
      <p>pelanggan@email.com</p>
    </div>
  </div>

  <div class="col-md-8">
    <div class="product-card">
      <h5>Informasi Akun</h5>
      <hr>
      <div class="mb-3">
        <label class="form-label">Nama Lengkap</label>
        <input type="text" class="form-control" value="Pelanggan Furniture">
      </div>
      <div class="mb-3">
        <label class="form-label">Email</label>
        <input type="email" class="form-control" value="pelanggan@email.com">
      </div>
      <div class="mb-3">
        <label class="form-label">No. Telepon</label>
        <input type="text" class="form-control" value="08123456789">
      </div>
      <button class="btn btn-furniture">Simpan Perubahan</button>
    </div>
  </div>
</div>

<div class="mt-4">
  <div class="product-card">
    <h5>Riwayat Pesanan</h5>
    <hr>
    <p class="text-muted">Belum ada riwayat pesanan.</p>
  </div>
</div>
@endsection
```

Halaman profil pengguna. Cara Kerja: `@csrf` token untuk keamanan, `Auth::user()` helper mengakses data user yang login.

Buat Routes

Routes/web.php

```

<?php

use Illuminate\Support\Facades\Route;
use Illuminate\Support\Facades\DB;

// Route utama aplikasi furniture
Route::get('/', function () {
    return view('furniture.home');
});

Route::get('/cart', function () {
    return view('furniture.cart');
});

Route::get('/profile', function () {
    return view('furniture.profile');
});

// Route untuk testing database (Tugas 12.2)
Route::get('/test-raw-sql', function () {
    try {
        // Raw SQL Query
        $products = DB::select('SELECT * FROM products LIMIT 5');
        return response()->json([
            'method' => 'Raw SQL Query',
            'data' => $products
        ]);
    } catch (\Exception $e) {
        return response()->json(['error' => $e->getMessage()]);
    }
});

Route::get('/test-query-builder', function () {
    try {
        // Query Builder
        $products = DB::table('products')->take(5)->get();
        return response()->json([
            'method' => 'Query Builder',
            'data' => $products
        ]);
    } catch (\Exception $e) {
        return response()->json(['error' => $e->getMessage()]);
    }
});

Route::get('/test-eloquent', function () {
    try {
        // Eloquent ORM
        $products = App\Models\Product::take(5)->get();
        return response()->json([
            'method' => 'Eloquent ORM',
            'data' => $products
        ]);
    } catch (\Exception $e) {
        return response()->json(['error' => $e->getMessage()]);
    }
});

// Route untuk insert data contoh (Tugas 12.2)
Route::get('/insert-raw', function () {
    $result = DB::insert('INSERT INTO products (name, price, category) VALUES (?, ?, ?)',
        ['Kursi Kantor', 850000, 'kursi']);
    return response()->json(['success' => $result, 'method' => 'Raw SQL']);
});

Route::get('/insert-querybuilder', function () {
    $result = DB::table('products')->insert([
        'name' => 'Meja Kerja',
        'price' => 1250000,
        'category' => 'meja',
        'created_at' => now(),
        'updated_at' => now()
    ]);
    return response()->json(['success' => $result, 'method' => 'Query Builder']);
});

Route::get('/insert-eloquent', function () {
    $product = new App\Models\Product();
    $product->name = 'Sofa Minimalis';
    $product->price = 2950000;
    $product->category = 'sofa';
    $result = $product->save();

    return response()->json([
        'success' => $result,
        'method' => 'Eloquent ORM',
        'id' => $product->id
    ]);
});

// Auth routes
Route::get('/login', [AuthController::class, 'showLogin'])->name('login');
Route::post('/login', [AuthController::class, 'login']);
Route::get('/register', [AuthController::class, 'showRegister'])->name('register');
Route::post('/register', [AuthController::class, 'register']);
Route::post('/logout', [AuthController::class, 'logout'])->name('logout');

// Order routes
Route::get('/order/raw', [OrderController::class, 'createOrderRawSQL']);
Route::get('/order/querybuilder', [OrderController::class, 'createOrderQueryBuilder']);
Route::get('/order/eloquent', [OrderController::class, 'createOrderEloquent']);

// Product routes dari ProductController
Route::get('/product/raw', [ProductController::class, 'insertRawSQL']);
Route::get('/product/querybuilder', [ProductController::class, 'insertQueryBuilder']);
Route::get('/product/eloquent', [ProductController::class, 'insertEloquent']);

```

Mendefinisikan semua endpoint aplikasi. Cara Kerja: Menerima HTTP request, memetakan ke closure atau controller method, mengembalikan response.

Output:

 **A Mitra Furniture**

Home [Keranjang](#) [Profil](#)

Selamat Datang di A Mitra Furniture

Aplikasi jual beli furniture terlengkap dan terbaik di Indonesia



Kursi Goyang
Rp 1.450.000

[+ Keranjang](#)



Sofa Modern
Rp 2.950.000

[+ Keranjang](#)



Meja Kayu Jati
Rp 1.350.000

[+ Keranjang](#)



Ranjang Minimalis
Rp 3.950.000

[+ Keranjang](#)

 **Status Database**

✔ **Migration Berhasil!**

Database sudah dibuat dengan 4 tabel sesuai Tugas Modul 12:

- **users** - Tabel pengguna
- **products** - Tabel produk furniture
- **carts** - Tabel keranjang belanja
- **orders** - Tabel pesanan

Tugas 12.1 ✔ - 4 File Migration sudah dibuat
Tugas 12.2 ✔ - 3 Metode Database sudah diimplementasi

[Test Raw SQL](#) [Test Query Builder](#) [Test Eloquent ORM](#)

Test Raw SQL

```
localhost:8000/test-raw-sql
Pretty-print ☒
{
  "method": "Raw SQL Query",
  "data": []
}
```

Test Query Vuilder

```
localhost:8000/test-query-builder
Pretty-print ☒
{
  "method": "Query Builder",
  "data": []
}
```

Test Eloquent ORM

```
localhost:8000/test-eloquent
Pretty-print ☒
{
  "method": "Eloquent ORM",
  "data": []
}
```

BAB III

PENUTUP

A. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, implementasi migration pada Laravel berhasil membangun empat tabel database (users, products, carts, orders) secara terprogram dengan kemampuan rollback. Tiga metode akses database (Raw SQL, Query Builder, Eloquent ORM) masing-masing memiliki keunggulan tersendiri: Raw SQL untuk performa maksimal, Query Builder untuk fleksibilitas, dan Eloquent untuk produktivitas pengembangan. Aplikasi "A Mitra Furniture" yang dibangun telah mengintegrasikan seluruh komponen MVC dengan baik, menunjukkan bahwa Laravel merupakan framework yang efektif untuk pengembangan aplikasi web modern dengan dukungan fitur database yang komprehensif.